

AI620: Fundamentals of Data Engineering

Project Guidelines

Amna Raza Khan, Shumaila Javed, and Hamna Aqeel
SBASSE, LUMS

Project Overview

Objective

- Implement a complete, reproducible data engineering pipeline: ingestion → storage → processing → serving.
- If your dataset/problem is small or straightforward, go deeper in one specialized track (e.g., advanced data quality, real-time streaming, etc.).
- **Teams:** 4 members

Core Requirements

1. Data Collection & Ingestion

- Use at least one real data source (API, scraping, or dataset).
- Ingestion must be automated (no manual uploads).
- Handle rate limits, authentication, and pagination where applicable.

2. Data Storage & Modeling

- Use a structured storage system (SQL/NoSQL/data lake).
- Define schema explicitly with proper data types and constraints.
- Justify design choice.

3. Data Transformation & Quality

- Handle missing values, type inconsistencies, duplicates, and outliers.
- Include at least one data validation or quality check (e.g., Great Expectations, custom validation).

- Implement idempotent transformations (re-running produces same output).
- Document transformation logic and assumptions.

4. Analytics or ML (Choose One Only)

- **Option A: Analytics**

- Meaningful aggregations or insights (e.g., trends, correlations, summaries)
- Exposed via dashboard or API with visualizations
- Include at least 3 distinct analytical queries

- **Option B: ML**

- Basic model (regression/classification/clustering)
- Integrated into the pipeline (not a standalone notebook)
- Include model evaluation metrics and feature importance analysis

5. Pipeline Orchestration

- Use a workflow orchestration tool (prefect).
- Define tasks and flows that automate your full pipeline (ingestion → transformation → loading → analytics/ML).
- Include scheduling (cron-based or interval-based) for automated recurring runs.
- Implement retry logic and failure handling for each task.
- Pipeline state and run history must be observable via a dashboard or logs.

6. Deployment & Serving

- Dockerized environment (required)
- One of the following serving methods:
 - API (FastAPI/Flask) with documented endpoints
 - Dashboard (Streamlit) with interactive elements
 - Scheduled batch output with automated delivery

7. Logging & Monitoring

- Capture logs of your pipeline runs or requests (model inference logs, pipeline run logs, errors).
- Show how you'd monitor or debug the solution if something goes wrong

Advanced Extensions (Choose One)

In addition to the complete pipeline, implement **one** advanced extension:

1. **Advanced Data Quality & Governance**
2. **Real-time Streaming & Distributed Processing**
3. **Advanced Orchestration & Scheduling**
4. **Production-grade Model Deployment (versioning, drift detection, advanced monitoring)**
5. **Scalable Batch Processing & Data Lakes (Spark, partitioning, big data best practices)**

Note: If your data is already large/complex, the full end-to-end pipeline itself will likely be enough of a challenge.

Sample Projects

- **Medium Author Success Predictor:** Extract key metrics from Medium's top authors to forecast the popularity potential of new articles or authors.
- **Spotify Song Recommender:** Aggregate Spotify listening data to deliver personalized music recommendations based on user preferences.
- **Movie and TV Series Metadata Platform:** Consolidate and enrich movie and TV metadata for efficient browsing, search, and discovery.
- **Intelligent Voice Assistant for Customer Support:** Transcribe and analyze customer calls in real-time to identify intents and streamline support operations.
- **Intelligent Data Lineage Tracker:** Develop a monitoring system that leverages LLMs to predict potential failures or bottlenecks in data pipelines based on historical performance data.

- **Predictive Data Pipeline Monitoring System:** Monitor pipeline metrics proactively to anticipate failures and maintain pipeline health.
- **Data Compliance Audit Assistant:** Create an assistant that utilizes LLMs to automatically review datasets and data processing activities for compliance with regulations such as GDPR or HIPAA.

Presentations

There will be **two presentations** during the project period. Both presentations are mandatory.

Checkpoint Presentation (Progress Review)

- **Timing:** Midway through the project period (date to be announced)
- **Duration:** 2-3 minutes per team
- **Purpose:** Demonstrate progress, receive feedback, and ensure you are on track

Checkpoint Presentation Requirements

Your checkpoint presentation must cover:

1. **Project Overview** (1 slide)
 - Problem statement and goals
 - Target dataset/data sources
 - Expected outcomes
2. **Architecture Design** (1 slide)
3. **Progress to Date** (1 slide)
 - Data collection/ingestion implementation status
 - Storage setup and schema design
 - Initial data processing or transformation
 - Any challenges encountered and how they were addressed
4. **Next Steps** (1 slide)
 - Remaining tasks and timeline

Final Presentation

- **Timing:** (date to be announced)
- **Duration:** 5-7 minutes presentation + 3-5 minutes Q&A
- **Purpose:** Showcase complete project, demonstrate working system, and present key learnings

Final Presentation Requirements

Your final presentation must cover:

1. **Project Introduction** (1 slide)
 - Problem statement and motivation
 - Target users or use cases
 - Success metrics
2. **System Architecture** (1 slide)
 - Final architecture diagram with all components
 - Technology stack and rationale
 - Data flow diagram
3. **Data Engineering Pipeline** (2 slides)
 - Data collection/ingestion strategy and implementation
 - Storage schema and modeling decisions
 - Data processing and transformation approach
 - Data quality validation methods
 - Orchestration and scheduling setup
4. **Analytics/ML Component** (1 slide)
 - Insights generated OR model performance metrics
 - Key findings or feature importance
 - How analytics/ML integrates with pipeline
5. **Deployment & Serving** (1 slide)
 - Deployment architecture (Docker, cloud)

- API endpoints OR dashboard features
 - How to run/access the system
6. **Logging & Monitoring** (1 slide)
- Logging implementation
 - Metrics tracked
 - Monitoring setup
7. **Live Demo** (1-2 minutes)
- Must show working system, demonstrate pipeline execution, show API calls or dashboard interactions
 - Highlight key features
8. **Challenges & Learnings** (1 slide)
- Technical challenges faced, how they were resolved and key takeaways
9. **Future Work** (1 slide)
- Potential improvements
 - Scalability considerations
10. **Team Contributions** (1 slide)
- Clear breakdown of each member's contributions

Project Submission Guidelines

1. Project Documentation (PDF File)

Please keep this file very concise (maximum 3 pages).

- **Team Details:** Roll numbers and 1-2 line contributions of each member
- **Project Description:** Title, goal, dataset, and domain/theme (1-2 paragraphs)
- **Architecture Diagram:** Use draw.io or similar
- **Schema Diagram:** Show data models and relationships
- **Pipeline Explanation:** Brief overview of each stage with tool justification

- **Deployment Link:** Link to deployed demo/interface (if publicly accessible)
- **Repository:** GitHub link to your repo/folder
- **AI Usage Disclosure:** Clearly specify use of AI in documentation (mandatory for visibility and judging)

2. Code (Python or other configuration files)

- **Structure:** Modular organization
 - Separate concerns: ingestion/, processing/, storage/, orchestration/, serving/
 - Configuration files for environment variables
 - Requirements.txt or environment.yml for dependencies
- **Documentation:** Well-commented, clear, and readable code
- **README:** Brief instructions on how to run the pipeline (one-command execution preferred)

3. Demo (Video File)

- **Duration:** 1-2 minutes (strict maximum)
- **Content:** Must show:
 - Pipeline execution (triggering the pipeline and observing it run)
 - System output (API calls with responses OR dashboard interactions)
 - Architecture explanation (walk through your design)
 - Code walkthrough (brief explanation of key components)
- **Format:** Voice over recommended
- **Note:** You can use this same demo for your in-class presentation to avoid live demo issues

Please note that each of the above is graded, including the DE stages mentioned, so missing anything will result in marks deduction. Please zip the above files in one project folder and upload them to LMS, in addition to providing us the link to your GitHub repo/folder.

Evaluation Criteria

Criterion	Weight
Ingestion Pipeline (automation, correctness, error handling)	15%
Data Transformation & Quality (cleaning, validation, idempotency)	15%
Storage & Data Modeling (schema design, justification)	10%
Analytics / ML Integration (insights quality or model performance)	15%
Orchestration & Scheduling (automation, retries, scheduling)	10%
Deployment & Reproducibility (Docker, one-command run)	15%
Logging & Monitoring (structured logs, metrics, observability)	10%
Documentation & Code Quality (clarity, modularity, README)	10%

Table 1: Project Evaluation Criteria

Timeline & Milestones

Milestone	Due Date	Deliverables
Checkpoint Presentation	TBD	2-3 minute presentation showing progress
Final Presentation	TBD	10-12 minute presentation + demo
Final Submission	TBD	All deliverables (code, report , demo)

Table 2: Project Timeline

Late Submission Policy

- Can not use Grace days for the project.

Academic Integrity

- You are allowed to use AI tools to understand concepts or debug syntax errors
- You are **required** to clearly declare any use of AI in your documentation (include which tools, what tasks, and how much)
- All team members must contribute meaningfully to the project
- Code and written content should be original work of the team
- Any violation of academic integrity will result in grade penalties

AI Usage Declaration Format

```
1 ## AI Usage Declaration
2 - Tool: ChatGPT
3 - Used for: Debugging syntax errors in FastAPI routes,
   understanding Airflow concepts
4 - Extent: No code generation, only debugging assistance
```